

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

федеральное государственное автономное образовательное
учреждение высшего образования



**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Инженерная школа информационных технологий и робототехники

09.04.04 «Программная инженерия»

Отделение информационных технологий

Отчет по лабораторной работе №6

по дисциплине «Управление разработкой промышленного программного обеспечения»

Выполнил:

Студент группы 8МП5Л

_____ Шаврин А.П.

Проверил:

К.т.н., доцент ОИТ

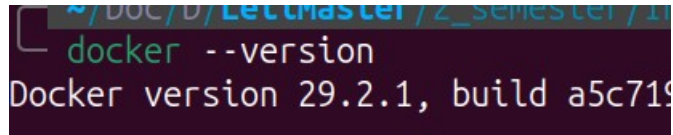
_____ Небаба С.Г.

Задание

Установить Docker на вашу операционную систему (или на виртуальную машину в вашей учебной аудитории), создать Dockerfile, собрать из него образ Docker Image и развернуть из образа Docker Image контейнер. Внутри контейнера должно быть ваше приложение или скрипт с любым дополнительным компонентом (например, это может быть Python), который будет установлен внутри образа и требуется для запуска приложения. Для компонентов можно использовать готовые образы из DockerHub. Запустить приложение.

Ход работы

Работа выполнялась на ОС Ubuntu 22.04. Docker уже был предустановлен ранее (рисунок 1).



```
~/DOC/D/LectMaster/2_semester/1
docker --version
Docker version 29.2.1, build a5c719
```

Рисунок 1. Версия докера

Создан файл app.py для минимального flask приложения со счетчиком и редисом в качестве кеша на рисунке 2.



```
1 import os
2 import redis
3 from flask import Flask
4
5 app = Flask(__name__)
6 cache = redis.Redis(
7     host=os.getenv("REDIS_HOST", "redis"),
8     port=int(os.getenv("REDIS_PORT", "6379")),
9 )
10
11 @app.route("/")
12 def hello():
13     count = cache.incr("hits")
14     return f"Hello from Docker! I have been seen {count} time(s).\n"
```

Рисунок 2. app.py.

Файл requirements.txt – зависимости представленные на рисунке 3.



```
flask
redis
```

Рисунок 3. requirements.txt

Затем был написан Dockerfile (рисунок 4). Выбран базовый образ python:3.12-slim – это образ, основанный на Debian, но содержащий только минимальный набор пакетов. Он имеет хорошую совместимость с Python-пакетами, особенно теми, которые требуют компиляции C-расширений (например, redis). Установка пакетов в slim происходит быстрее и с меньшим

количеством ошибок, чем в alpine, где часто нужно доустанавливать компиляторы и системные библиотеки. FLASK_APP=app.py – указывает Flask, какой файл запускать. Это обязательная переменная для команды flask run. Без нее Flask не знает, где находится приложение. FLASK_RUN_HOST=0.0.0.0 – заставляет Flask слушать все сетевые интерфейсы внутри контейнера. По умолчанию Flask слушает только 127.0.0.1 (локально внутри контейнера), что делает приложение недоступным с хоста. Установка 0.0.0.0 позволяет принимать соединения извне контейнера (через проброшенный порт). Эта переменная критически важна для корректной работы веб-приложения в Docker. Без этих переменных команда flask run либо не найдет приложение, либо приложение будет недоступно по сети.

```
1  name: lab5(PEP-8 Check)
2
3  on: [push, pull_request]
4
5  jobs:
6    lint:
7      runs-on: ubuntu-latest
8
9      steps:
10       - name: Checkout code
11         uses: actions/checkout@v4
12
13       - name: Set up Python
14         uses: actions/setup-python@v5
15         with:
16           python-version: '3.12'
17
18       - name: Install dependencies
19         run: |
20           python -m pip install --upgrade pip
21           if [ -f requirements.txt ]; then pip install -r requirements.txt; fi
22
23       - name: install pycodestyle
24         run: pip install pycodestyle
25
26       - name: Run pycodestyle
27         run: |
28           pycodestyle --max-line-length=99 **/*.py
29
```

Рисунок 3. check.yml

Затем была выполнена сборка образа контейнера (рисунок 4)

```
~/Doc/D/LetiMaster/2/I/lab6/src | 2_semester ?6 ..... ✓ | 09:21:29 PM
docker build -t flask-app .
[+] Building 3.0s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 246B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.12-slim 2.8s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:3d5ed973e45820f5ba5e46bd065bd88b3a504ff0 0.0s
=> => resolve docker.io/library/python:3.12-slim@sha256:3d5ed973e45820f5ba5e46bd065bd88b3a504ff0 0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 93B                                       0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt .                            0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY . .                                          0.0s
=> exporting to image                                             0.2s
=> => exporting layers                                              0.0s
=> => exporting manifest sha256:30c922e69e083199195eec96fa9b46a5f1f96a1cf617cfe1f3cf45e854edd061 0.0s
=> => exporting config sha256:ec6327fff900387af686248dc1d82a86060515008a4cdcdf339b911f8fc7f557 0.0s
=> => exporting attestation manifest sha256:84b70e3b37d08bd777d1c6f755bb6eb1ef0f619a0f8bdbb93508 0.0s
=> => exporting manifest list sha256:43267121bd23c3ef3a46dfee6a05cc779ab73540d854340d09344b76c0b 0.0s
=> => naming to docker.io/library/flask-app:latest                0.0s
=> => unpacking to docker.io/library/flask-app:latest              0.1s
```

Рисунок 4. Сборка образа.

Для взаимодействия контейнера с приложением и контейнера с redis создана пользовательская сеть, после был запущен контейнер redis в фоновом режиме в этой сети с базовым образом redis:alpine. Контейнер с приложением запущен с пробросом порта 8000 хоста на 5000 контейнера и передачей переменных окружения: --network mynet – подключение к той же сети, что и Redis. -e REDIS_HOST=redis – имя хоста Redis внутри сети соответствует имени контейнера. Результат запуска контейнеров представлен на рисунке 5.

```
~/Doc/D/LetiMaster/2/I/lab6/src | 2_semester ?6 ..... 3s | 09:21:36 PM
docker network create mynet
632d10e0194cd9c8eda3c196c60024ba6be6274495bf780864f6b171aa95c942

~/Doc/D/LetiMaster/2/I/lab6/src | 2_semester ?6 ..... 09:21:53 PM
docker run -d --name redis --network mynet redis:alpine
Unable to find image 'redis:alpine' locally
alpine: Pulling from library/redis
826f751c7f6d: Pull complete
cd9ebb746e5b: Pull complete
d67eeb8f9f36: Pull complete
4f4fb700ef54: Pull complete
cb2f87994157: Pull complete
d3f91c0b9ad5: Pull complete
da9f2e9b2d93: Download complete
5bf3e43e12be: Download complete
Digest: sha256:81b6f81d6a6c5b9019231a2e8eb10085e3a139a34f833dcc965a8a959b040b72
Status: Downloaded newer image for redis:alpine
22f92080053b341bb465d8328aa4d95e95d5bbbec9b63af61aa25d4c3ea3d72b

~/Doc/D/LetiMaster/2/I/lab6/src | 2_semester ?6 ..... 20s | 09:22:23 PM
docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
22f92080053b   redis:alpine "docker-entrypoint.s..." 8 seconds ago  Up 8 seconds  6379/tcp     redis

~/Doc/D/LetiMaster/2/I/lab6/src | 2_semester ?6 ..... 09:22:31 PM
docker run -d --name flask-web --network mynet -p 8000:5000 -e REDIS_HOST=redis -e REDIS_PORT=6379 flask-app
76117f272489d2987a04bd7746a0f618515804606b3cb4fa769213b4e374b606

~/Doc/D/LetiMaster/2/I/lab6/src | 2_semester ?6 ..... 09:22:42 PM
docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                                     NAMES
76117f272489   flask-app "flask run --debug"      3 seconds ago  Up 2 seconds  0.0.0.0:8000->5000/tcp, [::]:8000->5000/tcp  flask-web
22f92080053b   redis:alpine "docker-entrypoint.s..." 22 seconds ago  Up 22 seconds  6379/tcp     redis
```

Рисунок 5. Запуск контейнеров.

После чего был открыт браузер по адресу <http://localhost:8000>, на котором отображался счетчик приложения (рисунок 6).

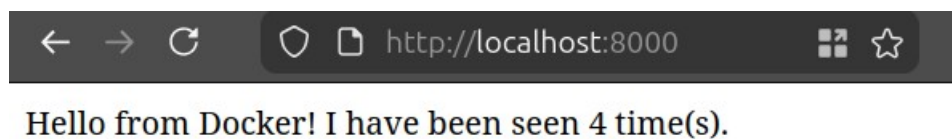


Рисунок 6. Проверка приложения.

Вывод

В ходе работы выполнена контейнеризация веб-приложения на Flask с Redis. Создан Dockerfile (базовый образ python:3.12-slim), собран образ, запущены два контейнера в общей сети Docker, настроено взаимодействие через переменные окружения. Приложение успешно работало, счетчик в Redis увеличивался. Освоены базовые принципы Docker: создание образов, управление сетями, проброс портов, передача переменных окружения, запуск и остановка контейнеров.